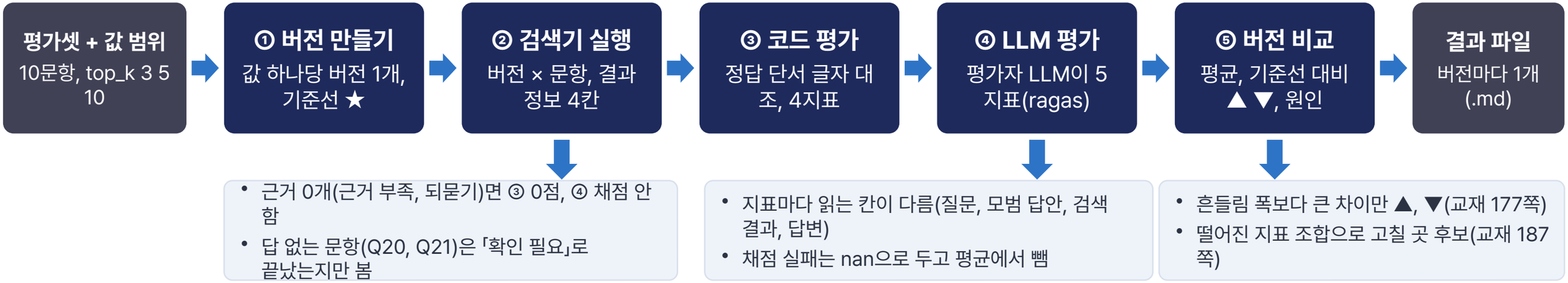
평가 워크플로우: 버전 비교 평가 구성안

하이퍼 파라미터 하나를 바꾼 버전마다 같은 평가셋으로 검색기를 돌리고, 코드 4지표와 LLM 5지표를 같은 잣대로 채점해 기준선과 비교함



단계	하는 일	판단 방식	핵심 코드
① 버전 만들기	값 목록으로 v01, v02, …를 만들고 나머지 설정은 기준선 값으로 고정함	코드 규칙	build_versions
② 검색기 실행	문항마다 run_workflow를 불러 질문, 모범 답안, 검색 결과, 답변을 한 행에 모음	검색기(LLM 포함)	make_result_row, clean_for_ragas
③ 코드 평가	적중 청크를 판정하고 하(Hit), 빠(Recall), 빨(MRR), 앞(nDCG)을 계산함	코드 계산(LLM 없음)	find_hits, retrieval_metrics, ndcg_at_k
④ LLM 평가	위, 정, 출, 관, 답 5지표를 평가자 LLM이 문항마다 판정함	평가자 LLM + 임베딩	build_metrics, score_row, @experiment
⑤ 버전 비교	버전별 매크로 평균, 기준선 대비 변화, 원인 후보를 내고 결과 파일을 씀	코드 규칙	summarize_versions, diagnose

출처: hands-on/solutions/ragas/evaluate_rag.py, core_eval.py, teacher/plan/ragas-개발계획서.md 4장, 교재 177, 188쪽

무단전재 및 배포 금지

평가 워크플로우: ① 버전 만들기 – 실제 코드

하이퍼 파라미터 하나의 값 목록을 버전 목록으로 바꿈. 기준선이 값 목록에 없으면 비교할 잣대가 없으므로 멈춤

실제 코드 (core_eval.py의 build_versions)

```
    def build_versions(param: str, values: list, baseline, supported: dict) → list[dict]:
①       if param not in supported:
            raise ValueError(f"지원하지 않는 하이퍼 파라미터: {param} → 지원 목록: {list(supported.keys())}")
②       if baseline is None:
            baseline = supported[param]
③       if baseline not in values:
            raise ValueError(f"기준선 값 {baseline} 이(가) 값 목록 {values} 에 없음 → --values에 기준선 값을 넣을 것")
        versions = []
        ...
        for value in values:
            ...
            number = len(versions) + 1
            version = {
④               "name": f"v{number:02d}",
                "param": param,
                "value": value,
⑤               "is_baseline": value == baseline,
            }
            versions.append(version)
        return versions
```

① 지원 목록 검사

재색인이 필요한 값(청크 크기, 임베딩 모델)은 지원 목록에 없으므로 여기서 멈춤

② 기준선 기본값

--baseline을 안 주면 검색기의 현재 값(top_k 5)이 기준선임

③ 기준선도 돌려야 함

같은 실행에서 기준선을 함께 채점해야 같은 잣대로 비교할 수 있음(교재 177쪽)

④ 버전 이름

v01, v02, ... 결과 파일 이름의 vNN이 됨. 같은 값은 한 번만 버전으로 만듦(...)

⑤ 기준선 표시

비교표에서 ★로 보이고, 흔들림 폭 반복 채점(--repeat)도 이 버전만 함

무단전재 및 배포 금지

평가 워크플로우: ② 검색기 실행 – 결과 정보 실제 코드

문항 1건 = 결과 정보 4칸(교재 188쪽). 질문과 모범 답안은 평가셋이, 검색 결과와 답변은 검색기가 채움. ragas에 넘기기 전에 회원번호와 인용 표시를 지움

실제 코드 (core_eval.py의 make_result_row + evaluate_rag.py의 clean_for_ragas)

```
    def make_result_row(item: dict, result: dict, question_for_ragas: str, answer_for_ragas: str) → dict:
        retrieved_contexts = []
        ...
①       for hit in result["evidence"]:
            retrieved_contexts.append(hit["text"])
            ...
        row = {
            "id": item["id"],
            ...
        }
②       row["user_input"] = question_for_ragas
③       row["reference"] = item["ground_truth"]
        row["retrieved_contexts"] = retrieved_contexts
④       row["response"] = answer_for_ragas
        return row

    def clean_for_ragas(question: str, answer: str) → tuple[str, str]:
⑤       question_clean = " ".join(retriever.MEMBER_ID_PATTERN.sub(" ", question).split())
        answer_clean = CITATION_PATTERN.sub("", answer).strip()
        return question_clean, answer_clean
```

❶ 검색 결과

검색기가 ⑤ 답변 생성에 넘긴 근거(리랭크 순서)의 본문만 모음. 근거 0개로 끝난 문항은 빈 목록

❷ 질문

회원번호를 지운 질문. 검색기가 답변 생성에 넘긴 것과 같은 문장

❸ 모범 답안

평가셋 ground_truth. 위, 정, 답 지표가 읽음

❹ 답변

인용 표시를 뺀 글. 출, 관, 답 지표가 읽음

❺ 정리 이유

평가자는 답변을 주장으로 쪼갤 때 「M-3001」과 「[D3-0019]」까지 주장에 넣는데 색인 본문에 없어 Faithfulness가 0이 됨(실측). 모든 버전에 같게 적용함

무단전재 및 배포 금지

평가 워크플로우: ③ 코드 평가 – 적중 청크 판정 실제 코드

검색 결과 청크마다 정답 단서가 글자 그대로 있는지 봄. 적중 청크 순위와 찾은 정답 근거를 돌려주고, 같은 정답 근거는 한 번만 셈

실제 코드 (core_eval.py의 find_hits)

```
    def find_hits(chunk_texts: list[str], relevance: list[dict]) → dict:
        hit_ranks = []
        found_evidence = set()
        ...
①       for rank, chunk_text in enumerate(chunk_texts, start=1):
            body = compact_text(chunk_text)
            numbers_in_chunk = []
②           for number, evidence in enumerate(relevance, start=1):
                for clue in evidence["clues"]:
③                   if compact_text(clue) in body:
                        numbers_in_chunk.append(number)
④                       found_evidence.add(number)
                        break
            ...
            if len(numbers_in_chunk) > 0:
⑤               hit_ranks.append(rank)
        return {
            "hit_ranks": hit_ranks,
            "found_evidence": found_evidence,
            ...
        }
```

① 순위마다

상위 k개 청크를 1위부터 봄. compact_text로 띄어쓰기와 줄바꿈을 지운 본문과 비교함(PDF 줄바꿈 차이)

② 정답 근거마다

정답 근거 번호(①이 1). 정답 근거 하나에 정답 단서(clues)가 여러 개일 수 있음

③ 글자 대조

단서도 공백을 지우고 본문에 그대로 있는지 봄. LLM 없이 같은 입력이면 같은 결과

④ 한 번만 셈

집합(set)이라 같은 정답 근거가 2위와 4위에 모두 있어도 하나로 셈(교재 181쪽 4위의 ①). break로 같은 근거의 다른 단서는 보지 않음

⑤ 적중 청크

정답 근거를 하나라도 담은 청크의 순위 목록. 4지표가 모두 이 목록을 씀

무단전재 및 배포 금지

평가 워크플로우: ③ 코드 평가 – 하, 빠, 빨 실제 코드

같은 적중 판정을 세 가지 질문으로 읽음. 하(Hit)는 답할 재료를 하나라도 건졌나, 빠(Recall)는 정답 근거를 빠짐없이 찾았나, 빨(MRR)은 첫 적중이 몇 위인가

실제 코드 (core_eval.py의 retrieval_metrics)

```
def retrieval_metrics(hits: dict, k: int) → dict:
    if hits["evidence_count"] == 0:
        return {"hit": None, "recall": None, "mrr": None}
    ranks_in_k = []
    for rank in hits["hit_ranks"]:
        if rank <= k:
            ranks_in_k.append(rank)
    if len(ranks_in_k) > 0:
        hit = 1
    else:
        hit = 0
    recall = len(hits["found_evidence"]) / hits["evidence_count"]
    if len(ranks_in_k) > 0:
        first_rank = ranks_in_k[0]
        mrr = 1 / first_rank
    else:
        mrr = 0.0
    return {"hit": hit, "recall": recall, "mrr": mrr}
```

❶ 답 없는 문항

정답 근거 0개면 Recall 분모가 0이라 계산하지 않고 None(교재 181쪽). 평균에서 뺌

❷ 상위 k개만

적중 순위 중 k 이하만 남김. k 밖의 적중은 이 지표에 들어가지 않음

❸ 하 Hit@k

상위 k개 안 적중 청크가 하나라도 있으면 1. 몇 개인지, 몇 위인지는 안 봄

❹ 빠 Recall@k

찾은 정답 근거 수 ÷ 전체 정답 근거 수. 세는 단위는 청크가 아니라 정답 근거(교재 180쪽)

❺ 빨 MRR

첫 적중 청크 순위의 역수. 2위면 0.50. 적중이 없으면 0. 여러 문항의 평균이 MRR(교재 181쪽)

무단전재 및 배포 금지

평가 워크플로우: ③ 코드 평가 – 앞(nDCG) 실제 코드

앞(nDCG@k) = DCG ÷ IDCG. 적중 청크가 위에 있을수록 점수가 크고, 이상적인 순서(관련 청크를 1위부터 채움)와 견줘 0 ~ 1로 맞춤

실제 코드 (core_eval.py의 ndcg_at_k)

```
def ndcg_at_k(hit_ranks: list[int], relevant_chunk_count: int, k: int) → float | None:
①   if relevant_chunk_count == 0:
        return None
    dcg = 0.0
    for rank in hit_ranks:
        if rank <= k:
②           dcg = dcg + rank_weight(rank)
③   ideal_count = min(relevant_chunk_count, k)
    idcg = 0.0
    for rank in range(1, ideal_count + 1):
④       idcg = idcg + rank_weight(rank)
⑤   return dcg / idcg
```

❶ 관련 청크 0개

IDCG가 0이 되어 나눌 수 없음(답 없는 문항) → None

❷ DCG

상위 k개 안 적중 칸의 순위 가중치 1 ÷ $\log_2$(순위 + 1)을 더함. 1위 1.00, 2위 0.63, 3위 0.50, 4위 0.43, 5위 0.39

❸ 이상적인 칸 수

관련 청크(색인 전체에서 정답 단서를 담은 청크)가 1위부터 몇 칸을 채우나. k칸이 최대

❹ IDCG

1위부터 ideal_count위까지의 가중치 합. 관련 청크 수 기준이라 nDCG가 1을 넘지 않음(181쪽 노트의 걱정이 사라짐)

❺ nDCG

181쪽 예: (0.63 + 0.43) ÷ (1.00 + 0.63 + 0.50 + 0.43) = 0.41. Q17 기준선 0.97(185쪽)

무단전재 및 배포 금지

평가 워크플로우: ④ LLM 평가 – 평가자와 지표 실제 코드

평가자 LLM(Groq Qwen)과 임베딩(Cohere)을 OpenAI 호환 주소로 만들고, ragas 0.4.3 collections 지표 5개를 만듦. 모범 답안이 있으므로 Precision은 WithReference

실제 코드 (evaluate_rag.py의 build_judge + core_eval.py의 build_metrics)

```
def build_judge(secrets: dict):
①   judge_client = AsyncOpenAI(
        api_key=secrets["GROQ_API_KEY"],
        base_url=GROQ_BASE_URL,
        ...
    )
②   llm = llm_factory(JUDGE_MODEL, client=judge_client, max_tokens=JUDGE_MAX_TOKENS)
    embedding_client = AsyncOpenAI(api_key=secrets["COHERE_API_KEY"],
base_url=COHERE_COMPAT_URL)
③   embeddings = OpenAIEmbeddings(client=embedding_client, model=EMBEDDING_MODEL)
    return llm, embeddings

def build_metrics(llm, embeddings) → dict:
    metrics = {
④       "context_recall": ContextRecall(llm=llm),
        "faithfulness": Faithfulness(llm=llm),
        "factual_correctness": FactualCorrectness(llm=llm),
    }
⑤   metrics["context_precision"] = ContextPrecisionWithReference(llm=llm)
⑥   metrics["answer_relevancy"] = AnswerRelevancy(llm=llm, embeddings=embeddings)
    return metrics
```

❶ 평가자 클라이언트

Groq의 OpenAI 호환 주소를 openai 패키지로 부름. 답변 LLM과 다른 모델(Qwen)이라 자기 채점이 아님

❷ ragas 0.4 방식

LangChain 래퍼는 0.4에서 deprecated라 llm_factory에 클라이언트를 직접 넘김

❸ 임베딩

인덱서와 같은 Cohere 모델을 OpenAI 호환 주소로. AnswerRelevancy만 씀

❹ 정, 출, 답

평가자 LLM만 있으면 되는 지표 3개. 답은 기본 F1

❺ 위

모범 답안이 있으므로 WithReference. 없으면 WithoutReference(답변 기준)

❻ 관

역질문과 원래 질문을 임베딩으로 비교하므로 embeddings도 넘김

무단전재 및 배포 금지

평가 워크플로우: ④ LLM 평가 – 한 행 채점(위, 정) 실제 코드

결과 정보 한 행을 지표 5개로 채점함. 지표마다 읽는 칸이 다르므로(교재 188쪽) 맞는 칸을 맞는 이름으로 넘김. 먼저 검색 지표 둘

실제 코드 (core_eval.py의 score_row)

```
    async def score_row(row: dict, metrics: dict) → dict:
①       question = row["user_input"]
        reference = row["reference"]
        contexts = row["retrieved_contexts"]
        answer = row["response"]
        ...
        scores = {}
②       scores["context_precision"] = await ascore_or_nan(
            metrics["context_precision"], "context_precision", errors,
③           user_input=question, reference=reference, retrieved_contexts=contexts)
④       scores["context_recall"] = await ascore_or_nan(
            metrics["context_recall"], "context_recall", errors,
            user_input=question, retrieved_contexts=contexts, reference=reference)
⑤       ...
        return scores
```

❶ 읽는 칸 4개

질문, 모범 답안, 검색 결과(청크 본문 목록), 답변. 188쪽 결과 정보 4칸 그대로

❷ 위 ContextPrecision

질문, 모범 답안, 검색 결과를 읽음. 청크마다 모범 답안에 쓸모 있나 1/0 → 순위대로 평균 정밀도

❸ 칸 이름

ragas가 정한 영어 이름을 그대로 씀. await는 평가자 호출이 끝날 때까지 기다림(비동기)

❹ 정 ContextRecall

질문, 검색 결과, 모범 답안을 읽음. 모범 답안을 주장으로 쪼개 검색 결과에 있나

❺ 나머지 3개

출, 관, 답은 다음 장. 실패한 지표는 ascore_or_nan이 사유를 적고 nan을 돌려줘 다음 지표로 감

무단전재 및 배포 금지

평가 워크플로우: ④ LLM 평가 – 한 행 채점(출, 관, 답) 실제 코드

답변 지표 셋. 출(Faithfulness)은 검색 결과 기준, 답(FactualCorrectness)은 모범 답안 기준이라 대조 대상이 다름(교재 185쪽)

실제 코드 (core_eval.py의 score_row)

```
async def score_row(row: dict, metrics: dict) → dict:
    question = row["user_input"]
    reference = row["reference"]
    contexts = row["retrieved_contexts"]
    answer = row["response"]
①   ...
    scores = {}
    ...
②   scores["faithfulness"] = await ascore_or_nan(
        metrics["faithfulness"], "faithfulness", errors,
        user_input=question, response=answer, retrieved_contexts=contexts)
③   scores["answer_relevancy"] = await ascore_or_nan(
        metrics["answer_relevancy"], "answer_relevancy", errors,
        user_input=question, response=answer)
④   scores["factual_correctness"] = await ascore_or_nan(
        metrics["factual_correctness"], "factual_correctness", errors,
        response=answer, reference=reference)
⑤   scores["errors"] = errors
    return scores
```

❶ 앞 장

위, 정 채점(검색 지표)

❷ 출 Faithfulness

질문, 답변, 검색 결과. 모범 답안은 읽지 않음 (근거 기준 환각 탐지)

❸ 관 AnswerRelevancy

질문과 답변만. 답변으로 역질문을 만들어 원래 질문과 임베딩 유사도

❹ 답 FactualCorrectness

답변과 모범 답안만. 검색 결과는 읽지 않음 (모범 답안 기준 사실 정확도, 기본 F1)

❺ 실패 사유

지표별 실패 사유 dict를 점수와 함께 돌려줌. 결과 파일의 「채점 실패」 칸이 됨

무단전재 및 배포 금지

평가 워크플로우: ④ LLM 평가 – 실험 실행 실제 코드

채점할 행을 ragas Dataset에 담고 @experiment 실험 함수를 arun으로 돌림(교재 178쪽). 모든 행을 한꺼번에 돌리므로 동시 채점 수를 세마포어로 제한함

실제 코드 (evaluate_rag.py의 run_llm_evaluation)

```
async def run_llm_evaluation(rows: list[dict], metrics: dict, experiment_name: str) → dict:
(1)   dataset = Dataset(name=experiment_name.replace("/", "_"), backend="inmemory")
      for row in rows:
          ...
      })
(2)   semaphore = asyncio.Semaphore(MAX_CONCURRENT_ROWS)
(3)   @experiment()
      async def ragas_eval(row):
          async with semaphore:
(4)           scores = await core.score_row(row, metrics)
          return {"id": row["id"], **scores}
(5)   experiment_result = await ragas_eval.arun(dataset, name=experiment_name)
      scores_by_id = {}
      for record in experiment_result:
          scores_by_id[record["id"]] = record
      return scores_by_id
```

❶ 평가셋 담기

한 문항 = 한 행(4칸 + id. 장표에서는 …로 줄임). 메모리(inmemory)에만 두고 파일은 만들지 않음

❷ 동시 채점 제한

세마포어 = 동시에 들어갈 수 있는 수를 세는 문지기. 2행씩 채점해 Groq 429(호출 제한)를 피함

❸ 실험 함수

데코레이터가 함수에 arun을 붙여 줌. 한 행(dict)을 받아 점수 행을 돌려줌

❹ 한 행 채점

앞 장의 score_row. 자리가 날 때까지 기다렸다가(async with) 들어감

❺ 문항 × 지표 표

arun이 모든 행을 동시에 돌리고 표로 모음. 실패한 행은 경고만 남기고 뺌

무단전재 및 배포 금지

평가 워크플로우: ⑤ 버전 비교 – 버전별 평균 실제 코드

버전마다 지표별 평균을 냄. 문항마다 점수를 먼저 내고 그 평균을 내는 매크로 평균(교재 181쪽)이라 정답 근거가 많은 문항에 무게가 쏠리지 않음

실제 코드 (core_eval.py의 summarize_versions)

```
def summarize_versions(rows_by_version: dict, metric_names: list[str], baseline_name: str, noise: dict) →
dict:
    summary = {}
①   for version_name, rows in rows_by_version.items():
        means = {}
        for metric in metric_names:
            values = []
            for row in rows:
②               values.append(row.get(metric))
③           means[metric] = mean_of(values)
④       summary[version_name] = {"mean": means, "delta": {}, "change": {}}
    ...
```

❶ 버전마다

rows는 그 버전의 문항 점수 dict 목록(키 = 지표 이름)

❷ 문항 점수 모으기

채점하지 않은 칸(답 없음, 근거 0개)은 None, 채점 실패는 nan

❸ 매크로 평균

제공 함수 mean_of가 None과 nan을 빼고 평균함. 남는 값이 없으면 None

❹ 버전 요약

delta(기준선과의 차이)와 change(▲, ▼, 변화 없음)는 다음 장에서 채움

무단전재 및 배포 금지

평가 워크플로우: ⑤ 버전 비교 – 기준선 대비 변화 실제 코드

기준선과의 차이를 흔들림 폭과 견줌. 점수 0.8 같은 고정 합격선 대신 기준선 대비 변화로 채택 여부를 정함(교재 187쪽)

실제 코드 (core_eval.py의 summarize_versions)

```
def summarize_versions(rows_by_version: dict, metric_names: list[str], baseline_name: str, noise: dict) -> dict:
    ...
①   baseline_means = summary[baseline_name]["mean"]
    for version_name, version_summary in summary.items():
        for metric in metric_names:
            mean = version_summary["mean"][metric]
            base = baseline_means[metric]
②           ...
③           delta = mean - base
            version_summary["delta"][metric] = delta
④           if delta > noise[metric]:
                version_summary["change"][metric] = "▲"
            elif delta < -noise[metric]:
                version_summary["change"][metric] = "▼"
            else:
                version_summary["change"][metric] = "변화 없음"
    return summary
```

❶ 기준선 평균

앞 장에서 낸 버전별 평균 중 기준선(★) 것

❷ 비교하지 않는 경우

기준선 자신은 「-」, 어느 한쪽이 None이면 「비교 불가」(장표에서 …로 줄임)

❸ 기준선 대비 차이

+면 나아짐. 결과 파일의 「▲ +0.125」 같은 숫자

❹ 흔들림 폭과 견줌

다시 채점만 해도 생기는 폭보다 크게 올랐을 때만 ▲, 크게 내렸을 때만 ▼, 그 사이는 「변화 없음」(교재 177쪽)

무단전재 및 배포 금지

평가 워크플로우: ⑤ 원인 찾기 – 실제 코드

기준선보다 떨어진(▼) 지표의 조합으로 검색, 순위, 생성 중 어디를 고칠지 고름(교재 187쪽 표). 지표 하나만 보면 놓침

실제 코드 (core_eval.py의 diagnose)

```
def diagnose(change: dict) → list[str]:
    reasons = []
①   if change.get("recall") == "▼":
        reasons.append("Recall@k ↓ (코드): 정답 근거를 놓침 → 청킹, 임베딩, Top-k를 봄")
    ...
②   context_recall_dropped = change.get("context_recall") == "▼"
    if context_recall_dropped:
        reasons.append("Context Recall ↓: 검색 정답률이 떨어짐 → 청킹, 임베딩, Top-k를 봄")
③   context_precision_dropped = change.get("context_precision") == "▼"
    if context_precision_dropped:
        reasons.append("Context Precision ↓: 검색 상위에 답이 많이 없음 → 리랭커, 하이브리드 비중을 봄")
    ...
④   if change.get("factual_correctness") == "▼":
        if context_recall_dropped:
            reasons.append("Factual Correctness ↓ + Context Recall ↓: 근거를 못 가져옴 → 청킹, 임베딩, Top-k
를 봄")
        else:
            reasons.append("Factual Correctness ↓, Context Recall 정상: 답변이 근거 일부만 담음 → 답변
프롬프트를 고침")
    ...
    return reasons
```

① 코드 지표

LLM 평가를 안 돌려도 나옴. Recall ↓는 정답 근거를 놓친 것 → 청킹, 임베딩, Top-k

② 정 ↓

검색 정답률 → 청킹, 임베딩, Top-k. change.get은 지표가 없으면 None이라 조건이 거짓

③ 위 ↓

상위에 답이 적음 → 리랭커, 하이브리드 비중

④ 답 ↓

정 ↓와 함께면 검색이 근거를 못 가져온 것, 정이 정상이면 답변이 근거 일부만 담은 것 (교재 185쪽 Q17) → 답변 프롬프트

무단전재 및 배포 금지